PR Review Design

Codebase Q&A and PR Review Assistant, Phase 3

Author	Todd Oppel
Repository	github.com/tpotechno/codebase-qa-assistant
Version	0.3.0 (Phase 3 complete)
Date	October 20, 2026
Status	Merged in PR #1; 47 automated tests passing

PDF version: pr_review_design.pdf · Previous phase: mcp_server_design.md

1. Summary

Phase 3 adds pull-request review. Three GitHub tools were added to the MCP server built in Phase 2: `get_pr`, `get_pr_diff`, and `post_review`. A new command, `npm run review -- owner/repo#123`, runs a review agent on the **Claude Agent SDK**. The agent reads the PR and its full diff, looks up surrounding code with the existing repo tools when it needs context, and returns a structured review: a verdict, a summary, and comments tied to exact lines, each tagged blocker, suggestion, or nit. The review is a draft by default. With `--post`, it is published to GitHub as a single comment-only review, with each inline comment on the line it refers to.

The agent never has the ability to write. It works with read-only tools, and publishing is a separate step that only the user can trigger.

2. Why

PR review is a task an AI assistant can genuinely help with: catching correctness and security problems early, and giving the human reviewer a first pass to react to. It is also a demanding test of the earlier phases. Review needs the repo tools from Phase 1 for context, the MCP server from Phase 2 as the tool contract, and new capabilities: reading GitHub, handling large diffs, and posting safely.

Phase 3 also moves orchestration to the Agent SDK on purpose. The `ask` command keeps its hand-written Messages API loop from Phase 1, and `review` uses the SDK. The repository now shows both approaches with the same tools, and the trade-off between them (Section 5).

Goals

- Review any GitHub PR the user can read, public or private, from one command.
- Produce comments anchored to lines GitHub accepts, so they can be posted inline.
- Keep the model away from any write capability. Posting is an explicit user action.
- Handle large PRs without losing content or exhausting the context window.
- Bound cost per review, and fail early and clearly when something is misconfigured.
- Test everything offline, with no API key or network.

Non-goals (this phase)

- Approving or requesting changes. Reviews are comment-only and advisory.
- Replying to or resolving existing review threads.
- Incremental re-review that reads prior comments and only looks at new commits.
- A web interface — Phase 4.
- Measuring review quality with an evaluation suite — Phase 5.

3. What was built

Architecture

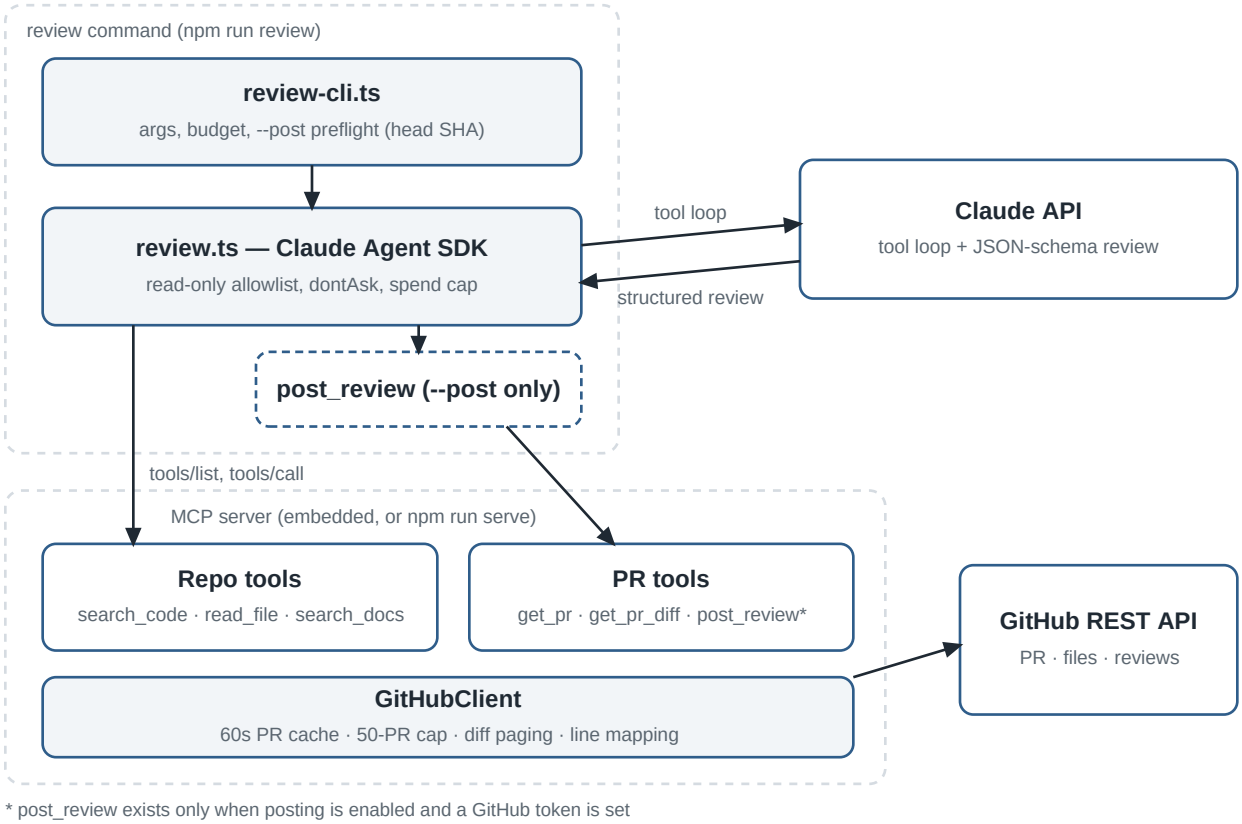

Figure 1. Runtime architecture. Dashed outlines group what runs together.

Components

File	Responsibility
`src/github.ts`	GitHub REST client: PR metadata, paged file list, posting reviews. Parses unified diffs to map new-file line numbers, formats PRs and diffs for the model, splits large diffs into pages, validates PR references, caches PRs, and neutralizes @mentions.
`src/mcp-server.ts`	Registers the three PR tools beside the Phase 2 repo tools. `post_review` is only registered when posting is enabled and a token is present.
`src/review.ts`	The review agent: Agent SDK options (tool allowlist, permissions, isolation, output schema, spend cap), the run loop, error handling, and Markdown formatting of the result.

File	Responsibility
`src/review-cli-lib.ts`	Testable command logic: argument parsing and the pre-posting checks.
`src/review-cli.ts`	The `review` command: wires configuration, the MCP server, the agent, and posting together.

Tools added to the MCP server

Tool	Input	Returns
`get_pr`	`pr` (owner/repo#123 or URL)	Title, description, author, branches, head commit, and changed files with line counts
`get_pr_diff`	`pr`, optional `path`, `page`	The diff with new-file line numbers in a gutter, paged at about 24,000 characters
`post_review`	`pr`, `body`, `comments[]`, optional `head_sha`	Publishes one COMMENT review; reports how many comments were inline and how many were moved to the body

4. How it works

Review flow

1. **Validate input.** The PR reference and budget are checked before anything else runs. A bad value exits immediately with a clear message.
2. **Get a tool server.** If `MCP_SERVER_URL` is set, the command uses that server. Otherwise it starts a private MCP server in-process on a random port with a one-time token, so a review needs one terminal.
3. **Preflight, only with `--post`.** Before spending anything, the command confirms the server offers `post_review` and reads the PR's current head commit. If either check fails, it stops. It fails closed: an unknown head commit is an error, not a skipped check.
4. **Run the agent.** The Agent SDK connects to the MCP server and runs the review loop. The agent calls `get_pr`, reads every page of `get_pr_diff`, and uses `search_code`, `read_file`, or `search_docs` when a change depends on code outside the diff.
5. **Return a structured review.** The SDK enforces a JSON schema: a verdict, a summary, and up to 25 comments, each with path, line, severity, and body. The command prints it as Markdown and can save it with `--out`.
6. **Post, only with `--post`.** The command calls `post_review` with the head commit recorded in step 3. The server refetches the PR. If the PR has new commits, posting is refused; otherwise the review is published.

Line-accurate diffs

GitHub only accepts inline comments on lines that appear in the diff, and one bad line makes it reject the whole review. So the diff the model sees has the new-file line number in a gutter on every added and unchanged line, and removed lines have no number:

```
@@ -2,18 +2,28 @@
    4   import { startHttpServer } from "./mcp-server.ts";
    5 + import { GitHubClient } from "./github.ts";
      - const legacy = true;
```

The same parser produces the set of commentable lines used to validate comments before posting. Blank lines and stray carriage returns are ignored, so they can't create phantom line numbers.

Large pull requests

- **Paging.** Diffs are packed into pages of about 24,000 characters; the header says which page this is and how to get the next.
- **Splitting.** A single file larger than a page is split at line boundaries into numbered parts (`[part 2 of 3]`) rather than truncated, so every line stays reachable.
- **Generated files.** Lockfiles, minified bundles, source maps, and snapshots show a one-line placeholder unless requested by `path`. On this project's own PR, the lockfile alone was about 10% of the diff.
- **Caching.** A PR is fetched once and reused for 60 seconds (10 minutes for the server a review starts itself), so paging a 4-page diff costs 2 GitHub requests instead of 8, and the agent sees one consistent snapshot. Failed fetches are not cached, expired entries are evicted, and at most 50 PRs are kept.

Posting

- **Comment-only.** Reviews are always posted with event `COMMENT`. The tool never approves or requests changes, which also means it works on your own PRs.
- **Nothing lost.** Comments that target a line outside the diff, or a file the PR didn't change, are moved into the review body under "Additional notes" instead of failing the review.
- **No stale anchors.** Posting compares the PR's current head commit with the one recorded before the review, and refuses if they differ.
- **No surprise pings.** @mentions in posted text are wrapped in code spans, so they render but notify nobody.

5. Agent SDK vs. the hand-written loop

Concern	`ask` — hand-written loop (Phases 1–2)	`review` — Claude Agent SDK (Phase 3)
Tool loop	Own code calls the Messages API, runs tools, sends results back	Provided by the SDK
MCP connection	Own MCP client translates tools to API format	Declared in options; the SDK connects
Permissions	Implicit: the loop only runs tools it was given	Explicit allowlist with `dontAsk` mode: anything else is denied
Output shape	Free text	JSON schema enforced, with retries
Limits	Turn limit	Turn limit and a hard spend cap in dollars

Concern	`ask` — hand-written loop (Phases 1–2)	`review` — Claude Agent SDK (Phase 3)
Cost reporting	Token counts	Dollar cost per run
Testing	Fake Messages client	Scripted SDK message stream
Best for	Learning, full control, minimal dependencies	Production agents where safety and limits matter

6. Safety and security

Risk	Mitigation
The model posts or changes something on its own	The agent's allowlist has only read-only tools; `post_review` isn't in it. Built-in Claude Code tools (shell, file edits) are disabled. Posting happens only in the command, after the review, with `--post`.
Extra tools leaking into the agent	Local Claude settings are ignored, MCP config is strict (only this server), and Claude Code session variables are stripped from the agent's environment. Found and fixed during live testing, when an unrelated browser tool appeared in the agent's tool list.
Prompt injection from PR content	The PR author controls what the model reads. Mitigations: read-only tools, diff-validated comments, neutralized @mentions, comment-only posting. The README advises reading a review before posting it on PRs from strangers.
Path manipulation in PR references	Owner and repo can't be `.` or `..`, and path segments are encoded, so URL normalization can't redirect an authenticated request to a different API path.
Comments landing on the wrong lines	Head-commit check before posting; line validation against the diff.
Runaway cost	Spend cap per review (`REVIEW_MAX_BUDGET_USD`, default $1), turn limit of 30, preflight checks before any spend, validated budget value.
Token exposure	The GitHub token lives only in the MCP server process. The embedded server uses a random one-time bearer token. Fine-grained token scoped to Pull requests (read and write) and Contents (read-only).

7. Key decisions and alternatives

Decision	Alternative considered	Why this choice
Agent returns a review; the command posts it	Give the agent `post_review`	Keeps every write behind an explicit human action, so a confused or manipulated model can't publish anything.
Structured output (JSON schema)	Parse comments out of free text	Reliable fields for path, line, and severity; the SDK retries until the output validates.
Line-number gutter in the diff	Let the model count lines from hunk headers	Models miscount; explicit numbers make inline comments land on the right lines.
Fold invalid comments into the body	Drop them, or let GitHub reject the review	Nothing the reviewer said is lost, and one bad line can't sink the whole review.

Decision	Alternative considered	Why this choice
Comment-only reviews	Approve / request changes based on the verdict	An AI review should inform the human decision, not make it.
Embedded MCP server by default	Always require `npm run serve`	One command, one terminal; a standalone server is still supported.
Keep `ask` on the hand-written loop	Move everything to the Agent SDK	Shows both approaches over the same tools; the Phase 1 loop stays as the minimal-dependency path.
Short PR cache	No cache, or a long-lived cache	Cuts GitHub requests (60 per hour without a token) and keeps one snapshot per review, while posting always refetches.

8. Dogfooding: the assistant reviewed its own PR

Phase 3 was delivered as PR #1, and the review command was run against it. Every finding was checked against the code before it was fixed.

Round 1 (posted to GitHub; all 6 comments fixed, replied to, and resolved):

1. A single file larger than a page was silently truncated. Now split into parts.
2. A trailing newline could create a phantom commentable line. Now ignored.
3. Each diff page refetched the PR. Now cached; the server also kept creating a new client per request, which was fixed too.
4. New commits during a review could misplace comments. Now refused at posting.
5. A blank or malformed budget value broke the spend cap. Now validated.
6. The README's hardcoded test count went stale. Removed.

Round 2 (draft; all 8 findings fixed):

1. The head-commit check could silently switch off. Now fails closed.
2. A long review could mix PR snapshots. A longer cache was added for the review's own server.
3. **Security:** `../..#1` could redirect an authenticated request. Dot segments are now rejected, and path segments are encoded.
4. The PR cache never evicted entries. Now evicts and caps at 50.
5. Test data used an unrealistic commit ID. Now 40 characters.
6. The command's own logic had no tests. Extracted into `review-cli-lib.ts` and tested.
7. A prompt-injected PR could make a review @mention people. Mentions are now neutralized.
8. `--out --post` treated `--post` as a file name. Now rejected.

The first round found real bugs; the second found mostly hardening, plus one real security issue. The project stopped there, with no blockers or security issues remaining, rather than chasing further rounds of nits.

Found by live testing before review: the agent's tool list included an unrelated MCP server inherited from the environment; an invalid API key surfaced as a vague error because the SDK reports it as `success` with `is_error`; and lockfiles were wasting about 2,000 tokens per diff read. All three were

fixed. A merge conflict in `package-lock.json` also silently blocked CI on the PR until it was resolved.

9. Testing

47 tests run in a few seconds with no API key or network, on every push in GitHub Actions.

Area	What is verified
GitHub client	PR references (including rejected dot segments), file paging past 100 files, error messages with hints, the PR cache (sharing, bypass, eviction, cap, failures not cached), posting (COMMENT only, folding, head-commit refusal, @mention neutralizing)
Diff handling	Line mapping, trailing-newline and CRLF artifacts, large-file splitting with every line reachable, generated-file placeholders, page size limits
MCP server	PR tools over real HTTP against a fake GitHub, `post_review` registered only with posting enabled and a token, stale or malformed `head_sha` rejected
Review agent	Read-only allowlist, `dontAsk`, isolation settings, spend cap, schema output, tool-call reporting, disconnected server, budget exhaustion, missing output, `is_error` results
Command logic	Argument parsing (order, missing values, unknown options), the pre-posting checks (server can't post, PR unreadable, head commit unknown)

A fake GitHub API and a scripted Agent SDK message stream make the tests deterministic. Live checks against PR #1 confirmed that real line numbers match the real file, and that the agent sees exactly the intended tools.

10. Configuration and usage

Variable	Default	Purpose
`GITHUB_TOKEN`	unset	Fine-grained token for private repos, higher rate limits, and `--post`
`GITHUB_ALLOW_POST`	`false`	Registers `post_review` on a standalone `npm run serve` server; the review command's own server enables it with `--post`
`REVIEW_MAX_BUDGET_USD`	`1`	Spend cap per review
`MCP_SERVER_URL`	unset	Use a standalone server instead of the embedded one
`CLAUDE_MODEL`	`claude-sonnet-5-5`	Model for the review agent

```
# Draft review (nothing posted)
npm run review -- tpotechno/codebase-qa-assistant#1

# Save it as Markdown too
npm run review -- owner/repo#123 --out review.md

# Publish to GitHub as a comment-only review
npm run review -- owner/repo#123 --post
```

A review of PR #1 (about 1,200 changed lines) took 7 turns, 31 seconds, and $0.14.

11. Limitations and next steps

- `--post` **runs a fresh review.** It doesn't publish an earlier draft, so the posted review can differ from the one you read. Next: save the structured review and post it with a separate command.
- **Whole-PR reviews.** Each run reviews the entire PR and doesn't read existing comments, so re-reviews can repeat resolved points. Next: review only new commits and skip resolved threads.
- **New-file lines only.** Comments can target added and unchanged lines, not removed ones.
- **No thread management.** The tool can't reply to or resolve threads; that was done manually for PR #1.
- **Review quality isn't measured.** Next (Phase 5): an evaluation set of PRs with known issues, scoring what the reviewer finds and misses.
- **Phase 4:** an Angular web interface for asking questions and running reviews, with streaming output and a visible tool-call trace.